Distributed Training of Large Language Models: Architectures, Parallelism, and Communication Challenges

Andrew Photinakis
Rochester Institute of Technology
acp7795@rit.edu
Rochester, New York, USA

I. Abstract

Scaling of Large Language Models to hundreds of billions or trillions of parameters has fundamentally invalidated single-device training paradigms. While model capacity remains directly correlated with downstream task performance, hardware advancement has proven stagnant relative to exponential growth of neural architecture memory footprints. Distributed deep learning landscape features a sharp architectural divide between automated search engines and manual heuristic frameworks. Manual 3D parallel systems require deep domain expertise to hardcode partitioning rules, frequently leading to sub-optimal resource allocation across heterogeneous interconnect topologies. Automated compilers and auto-tuners attempt to navigate these multi-dimensional search spaces algorithmicly; however, systems that isolate parallelism from memory footprint optimizations hit severe performance ceilings or physical memory limits. Scaling Large Language Models (LLMs) past single-accelerator High Bandwidth Memory (HBM) limits requires co-designing distributed software schedules with physical network topologies. This systematic review evaluates distributed LLM training paradigms, contrasting static 3D parallelism baselines against dynamic memory-parallelism systems like Mist and WAN-oriented frameworks like PGCS. We analyze state sharding mechanisms, tensor splitting, and pipeline scheduling, quantifying exact memory footprints and collective transmission byte volumes. Guided by the Scalability, Efficiency, and Reliability (SER) framework, we deconstruct multi-stream overlap templates, symbolic performance modeling, and hierarchical auto-tuning. Finally, we critique baseline architectural assumptions under severe network bottlenecks and formulate key open research problems for resilient foundation model infrastructure.

II. Introduction

The strategic imperative of modern Artificial Intelligence centers on the continuous scaling of foundation models, where parameter counts exceed hundreds of billions to trillions. However, single-accelerator compute throughput and High Bandwidth Memory (HBM) capacities have failed to maintain parity with parameter growth. A state-of-the-art accelerator, such as an NVIDIA H100 GPU with 80GB of HBM, cannot store the stateful memory of a multi-billion parameter model during training, creating a sever memory wall. For instance, training ultra-scale foundation models

like LLaMa-3.1-405B demands an aggregate computational effort of 30.84 million GPU hours on H100 accelerators. Domain fine-tuning or continuous pre-training consumes over 2,000 H100 GPU hours per billion tokens.

To contextualize cluster-level scaling efficiency, distributed systems engineering evaluates total model throughput using the governing cluster speedup equation established in survey literature:

Maximizing total training speed requires scaling across multi-rack data center networks and multi-site Wide Area Networks (WANs). However, hardware physical infrastructure ceilings in centralized facilities impose strict boundaries on cluster expansion.

1) Power Delivery Constraints: Substation capacities cap maximum power draw per server hall, limiting rack density
2) Rack Space & Thermal Cooling Limits: Liquid and forced-air cooling thresholds cap dissipation capacity per thermal zone.
3) Interconnect Topology Ceilings: Physical cable routing, switch port densities, and bisection bandwidth limits constrain cross-rack scalability.

The core distributed computing challenge lies in balancing three interdependent variables: per-device memory footprint, inter-node network communication latency, and micro-architectural compute efficiency. Minimizing per-device memory usage through aggressive state sharing, parameter offloading, or activation recomputation introduces extra collective network transfers or redundant forward pass calculations. Conversely, minimizing communication overhead by replicating states causes increased Out-Of-Memory (OOM) failures. This paper systematically investigates how modern distributed architectures resolvethese fundamental engineering trade-offs. We analyze foundational 3D parallelism taxonomies, examine state-of-the-art execution frameworks and synthesize broad survey literature. Through rigorous architectural critiques, empirical telemetry evaluations, and quantitative memory accounting, this paper maps the systemic landscape of next-generation machine learning infrastructure. To establish an analytical baseline, we first construct the fundamental mathematical taxonomy of memory allocations and parallelization strategies in stateful distributed training.

III. Distributed Architectures & Algorithms

A. Training Resource Requirements

Stateful pre-training of Transformer-based LLMs requires intense memory accounting to prevent OOM faults. The total memory consumed on a training accelerator during an iterative training step comprises four distinct operational components: state model states, activation tensors, Key-Value (KV) cache structures, and transient intermediate CUDA kernel workspaces. For standard mixed-precision training using an FP16/BF16 representation paired with an FP32 Adam optimizer, the memory footprint required for static model states is striclty 16 byters per parameter (Φ), structured as follows:

1) Model Parameters (**2Φ** bytes): Stored in 16-bit floating-point format (FP16/BF16) for high-throughput tensor

core matrix computations during forward and backward execution passes.

2) Gradients (**2Φ** bytes): Stored in 16-bit format (FP16/BF16) generated during backpropagation.
3) Master Model Parameters (**4Φ** bytes): Maintained in 32-bit single-precision floating-point (FP32) to ensure numerical stability during weight updates against gradient underflow or overflow.
4) Adam Optimizer States (**8Φ** bytes): Comprising the FP32 first moment vector (momentum, 4Φ bytes) and the FP32 second moment vector (uncentered variance, 4Φ bytes).

$$\text{Memory}_{\text{static}} = 2\Phi_{\text{params}} + 2\Phi_{\text{grads}} + 4\Phi_{\text{master}} + 8\Phi_{\text{Adam}} = 16\Phi \text{ bytes.}$$

Beyond static model states, volatile forward-pass activation tensors, such as intermediate linear layer outputs, attention matrix mapers, and LayerNorm inputs, scale directly with batch size, sequence length, and hidden dimensions. Formally, the activation memory footprint $\text{Memory}_{\text{act}}$ per Transformer layer without recomputation is expressed as:

$$\text{Memory}_{\text{act}} \propto b \cdot s \cdot h \cdot L \cdot \left(1 + \frac{2 \cdot a}{h}\right)$$

Where b represents microbatch size, s is sequence length, h is hidden dimension size, L is layer depth, and a denotes the attention head count. Selective Activation Checkpointing (CKPT) reduces this volatile footprint by stashing only key operator boundary tensors while recomputing cheap intermediate element-wise activations during the backward pass.

B. Data Parallelism and State Sharding

Distributed Data Parallelism (DDP) replicates full model copies across N GPU workers, partitioning the global training dataset into distinct local batch slices. While conceptually straightforward, canonical DDP requires every worker GPU to retain the full 16Φ static memory footprint, rendering it unviable for multi-billion parameter models on standard hardware. To eliminate redundant memory storage across data-parallel ranks, state-sharded paradigms exemplified by the Zero Redundancy Optimizer (ZeRO) and Fully Sharded Data Parallel (FSDP) partition model states across the data-parallel degree DP:

1) ZeRO-Stage 1: Shards the 12-byte FP32 Adam optimizer states (4Φ master weights + 8Φ momentum/variance) across DP devices, reducing memory per worker to $4\Phi + \frac{12\Phi}{DP}$ bytes. Communication volume remains identical to standard DDP (one gradient All-Reduce per iteration).
2) ZeRO-Stage 2: Shards both optimizer states and FP16 gradients across DP devices. The gradient reduction primitive switches from an All-Reduce to a Reduce-Scatter, reducing per-device memory to $2\Phi + \frac{14\Phi}{DP}$ bytes without increasing overall network traffic.
3) ZeRO-Stage 3 (FSDP): Shards all optimizer states, gradients, and FP16 parameters across DP devices. Per-device static memory footprint drops to $\frac{16\Phi}{DP}$ bytes.

To establish network-byte transmission trade-offs, let $|P|$ represent the total parameter size in bytes, $|G|$ represent the total gradient size in bytes, and d_{DP} represent the data-parallel degree. Assume that $|P| = |G| = |\Phi|$. Then,

$$\text{Volume}_{\text{DDP}} = 2\left(\frac{d_{\text{DP}} - 1}{d_{\text{DP}}}\right)|G|, \tag{1}$$

$$\text{Volume}_{\text{ZeRO-2}} = \left(\frac{d_{\text{DP}} - 1}{d_{\text{DP}}}\right)|G| \qquad \text{(Reduce-Scatter)}, \tag{2}$$

$$\begin{aligned}\text{Volume}_{\text{ZeRO-3}} &= \underbrace{2\left(\frac{d_{\text{DP}} - 1}{d_{\text{DP}}}\right)|P|}_{\text{forward and backward All-Gather}} + \underbrace{\left(\frac{d_{\text{DP}} - 1}{d_{\text{DP}}}\right)|G|}_{\text{backward Reduce-Scatter}} \\ &= 3\left(\frac{d_{\text{DP}} - 1}{d_{\text{DP}}}\right)|\Phi|. \end{aligned} \tag{3}$$

C. Memory and Communication Costs

Let Φ denote the number of model parameters and let D denote the data-parallel degree. Under mixed-precision Adam training, static model state consists of FP16 parameters, FP16 gradients, FP32 master parameters, and two FP32 Adam moment states. Without sharding, the resulting static memory requirement is

$$\begin{aligned} M_{\text{static}} &= \underbrace{2\Phi}_{\text{FP16 parameters}} + \underbrace{2\Phi}_{\text{FP16 gradients}} + \underbrace{4\Phi}_{\text{FP32 master parameters}} + \underbrace{8\Phi}_{\text{FP32 Adam states}} \\ &= 16\Phi \text{ bytes}. \end{aligned} \tag{4}$$

This estimate excludes activations, temporary workspaces, communication buffers, and memory fragmentation.

Let $|P|$ denote the parameter-tensor size in bytes and let $|G|$ denote the gradient-tensor size in bytes. For ring-based collective operations across N devices, the approximate communication volume per device is shown in Table I.

TABLE I

PER-DEVICE COSTS OF RING-BASED COLLECTIVE OPERATIONS

Primitive	**Operation**	**Communication volume**
All-Reduce	Reduces a tensor and distributes the complete result to every device.	$2\left(\frac{N-1}{N}\right)\|X\|$
Reduce-Scatter	Reduces a tensor and distributes one result shard to each device.	$\left(\frac{N-1}{N}\right)\|X\|$
All-Gather	Collects tensor shards and reconstructs the complete tensor on every device.	$\left(\frac{N-1}{N}\right)\|X\|$

The memory and communication costs of DDP and the three ZeRO stages are summarized in Table II. The static-memory expressions use the 16Φ-byte model described above.

TABLE II

PER-DEVICE MEMORY AND COMMUNICATION COSTS OF DATA-PARALLEL STRATEGIES

Strategy	**State distribution**	**Static memory**	**Communication per iteration**
DDP	All model states are replicated.	16Φ	$2\left(\frac{D-1}{D}\right)\lvert G\rvert$
ZeRO-1	Optimizer states are sharded.	$4\Phi + \frac{12\Phi}{D}$	$2\left(\frac{D-1}{D}\right)\lvert G\rvert$
ZeRO-2	Optimizer states and gradients are sharded.	$2\Phi + \frac{14\Phi}{D}$	$\left(\frac{D-1}{D}\right)(\lvert G\rvert + \lvert P\rvert)$
ZeRO-3	Parameters, gradients, and optimizer states are sharded.	$\frac{16\Phi}{D}$	$\left(\frac{D-1}{D}\right)(2\lvert P\rvert + \lvert G\rvert)$

Assuming that parameters and gradients have equal tensor sizes, $|P| = |G| = B$, the ZeRO-2 communication volume becomes

$$V_{\text{ZeRO-2}} = 2\left(\frac{D-1}{D}\right)B, \tag{5}$$

whereas the ZeRO-3 communication volume becomes

$$V_{\text{ZeRO-3}} = 3\left(\frac{D-1}{D}\right)B. \tag{6}$$

Therefore,

$$\frac{V_{\text{ZeRO-3}}}{V_{\text{ZeRO-2}}} = \frac{3}{2} = 1.5. \tag{7}$$

Under these assumptions, ZeRO-3 communicates 1.5× as many bytes per device per iteration as DDP or ZeRO-2. This additional communication results from reconstructing parameter shards through All-Gather operations during both the forward and backward passes.

D. Tensor Parallelism: Intra-Layer Model Parallelism

Tensor parallelism (TP), popularized by Megatron-LM, partitions the computations and weight matrices of an individual neural-network layer across multiple accelerators. Unlike pipeline parallelism, which assigns complete groups of layers to different devices, tensor parallelism divides operations within each Transformer layer. It is commonly applied to the weight matrices of the multi-head attention and multilayer perceptron (MLP) modules.

Consider an MLP block with input X, intermediate activation H, and output Z:

$$H = \text{GeLU}(XW_1), \qquad Z = HW_2. \tag{8}$$

Let T denote the tensor-parallel degree. Megatron-style tensor parallelism partitions W_1 by columns and W_2 by rows:

$$W_1 = \begin{bmatrix} W_{1,1} & W_{1,2} & \cdots & W_{1,T} \end{bmatrix}, \qquad W_2 = \begin{bmatrix} W_{2,1} \\ W_{2,2} \\ \vdots \\ W_{2,T} \end{bmatrix}. \tag{9}$$

The two partitioned linear operations proceed as follows:

1) **Column-parallel linear layer.** The input X is replicated across all T tensor-parallel ranks. Rank i stores the column partition $W_{1,i}$ and independently computes

$$H_i = \mathrm{GeLU}(XW_{1,i}). \tag{10}$$

The complete intermediate activation is logically partitioned as

$$H = \begin{bmatrix} H_1 & H_2 & \cdots & H_T \end{bmatrix}. \tag{11}$$

Because the following row-parallel layer consumes the corresponding activation partition directly, the complete H tensor does not need to be reconstructed between the two linear operations.

2) **Row-parallel linear layer.** Rank i stores the row partition $W_{2,i}$ and consumes the corresponding activation partition H_i. Each rank computes a partial output

$$Z_i = H_i W_{2,i}. \tag{12}$$

The partial outputs are summed across all tensor-parallel ranks using an All-Reduce collective:

$$Z = \mathrm{AllReduce}_{\mathrm{sum}}\left(Z_1, Z_2, \ldots, Z_T\right) = \sum_{i=1}^{T} Z_i. \tag{13}$$

This paired column-and-row partitioning reduces the amount of parameter memory and computation assigned to each accelerator while avoiding reconstruction of the intermediate MLP activation. However, the row-parallel operation places an All-Reduce collective directly on the critical execution path of the Transformer layer. Similar collective operations occur in tensor-parallel attention modules. Consequently, tensor parallelism is sensitive to network latency and bandwidth because these collectives are invoked repeatedly during both the forward and backward passes.

Higher-dimensional variants, including 2D, 2.5D, and 3D matrix-multiplication schemes, distribute tensors across multidimensional device meshes. These methods can reduce the per-device communication volume at larger tensor-parallel degrees, but they introduce more complicated partitioning, placement, and collective-communication

schedules. Because of its frequent fine-grained communication, tensor parallelism is generally deployed within high-bandwidth communication domains, such as GPUs connected through NVLink and NVSwitch. Extending tensor parallelism across slower inter-node links can place collective-communication latency on the critical path and reduce the benefit of distributing the computation.

E. Pipeline Parallelism: Inter-Layer Model Parallelism

Pipeline parallelism (PP) partitions a model along its layer dimension into p pipeline stages. Each stage contains a contiguous group of layers and is assigned to a different accelerator or compute node. A training batch is divided into m microbatches, which move through the stages as a pipeline. After a stage finishes processing one microbatch, it can begin processing the next while downstream stages continue working on earlier microbatches.

Adjacent stages exchange activation tensors during the forward pass and activation gradients during the backward pass. Pipeline parallelism therefore reduces the number of model parameters stored by each stage, but introduces point-to-point communication, pipeline bubbles, activation-storage requirements, and potential load imbalance between stages.

1) Pipeline Execution Schedules

Two common pipeline schedules are the fill-drain and one-forward-one-backward schedules.

GPipe fill-drain schedule: During the fill phase, all m microbatches execute their forward passes through the pipeline. Backpropagation begins only after the final microbatch completes its forward pass. The pipeline then drains as the backward passes execute in reverse order. This schedule is synchronous and avoids weight staleness, but the forward phase creates a large number of outstanding activations. Activation checkpointing can reduce this memory requirement by retaining selected activations and recomputing others during the backward pass.

One-forward-one-backward schedule: A one-forward-one-backward (1F1B) schedule interleaves forward and backward computations after an initial warm-up period. In steady state, a stage alternates between processing one forward microbatch and one backward microbatch. PipeDream popularized this form of interleaved scheduling, although different systems use different synchronization and weight-version policies. Compared with fill-drain scheduling, 1F1B limits the number of outstanding activations to a quantity proportional to the pipeline depth. Some asynchronous implementations may introduce weight staleness, whereas flush-based implementations preserve synchronous training semantics.

Under an idealized model in which all p stages have equal execution times and communication overhead is ignored, processing m microbatches requires $m + p - 1$ pipeline time slots. The ideal pipeline utilization is therefore

$$\eta_{\mathrm{PP}} = \frac{m}{m + p - 1}. \tag{14}$$

The fraction of execution time lost to pipeline bubbles is

$$f_{\text{bubble}} = 1 - \eta_{\text{PP}} = \frac{p-1}{m+p-1}. \tag{15}$$

The bubble overhead relative to useful computation can alternatively be expressed as

$$r_{\text{bubble}} = \frac{p-1}{m}. \tag{16}$$

Increasing the number of microbatches reduces the relative bubble overhead. However, it may increase scheduling complexity and activation-memory pressure. In practical systems, unequal stage execution times, communication delays, and hardware heterogeneity can reduce utilization below the ideal value in Equation 14.

F. Hybrid Parallelism and Specialized Extensions

Large-scale training systems commonly combine data, tensor, and pipeline parallelism into a three-dimensional parallel configuration. Let d_{DP}, d_{TP}, and d_{PP} denote the data-, tensor-, and pipeline-parallel degrees, respectively. The total number of accelerators is

$$N_{\text{total}} = d_{\text{DP}} d_{\text{TP}} d_{\text{PP}}. \tag{17}$$

Within this organization, tensor-parallel groups divide individual layer operations, pipeline-parallel groups divide the model into layer stages, and data-parallel groups process different training samples. The configuration must be selected with respect to model dimensions, memory capacity, network topology, batch size, and communication cost. Increasing one parallel dimension can reduce a particular resource constraint while increasing communication or scheduling overhead elsewhere.

1) Sequence Parallelism

Sequence parallelism (SP) partitions activations along the input-sequence dimension across the members of a tensor-parallel group. It is particularly useful for operations such as layer normalization and dropout, which are otherwise replicated across tensor-parallel ranks.

Let X be an activation tensor with sequence length s. Under sequence parallelism, X is partitioned as

$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_{d_{\text{TP}}} \end{bmatrix}, \qquad X_i \in \mathbb{R}^{(s/d_{\text{TP}}) \times h}, \tag{18}$$

where h is the hidden dimension. Each rank stores and processes only its local sequence partition. Implementations can replace selected tensor-parallel All-Reduce operations with paired Reduce-Scatter and All-Gather operations. This preserves a comparable aggregate communication volume while reducing the peak activation memory stored by each rank. Sequence parallelism is therefore useful for models trained with long input sequences.

2) Expert Parallelism

Expert parallelism (EP) is designed for sparse mixture-of-experts (MoE) architectures. Instead of replicating every expert on every device, the E expert subnetworks are distributed across an expert-parallel group. A learned router assigns each input token to one or more experts.

If expert e is stored on a remote device, tokens assigned to that expert must be transferred to the owning device. This dispatch is commonly implemented using an All-to-All collective. After the selected experts process the tokens, a second All-to-All operation returns the results to their original ranks. Expert parallelism reduces the memory and computation required per device, but its performance depends on routing balance and All-to-All network performance. Uneven token assignments can create expert hot spots, communication imbalance, and accelerator idle time.

G. Communication Operations and Hardware Interconnects

The effectiveness of a distributed training strategy depends on the physical topology connecting its accelerators. Different parallelism dimensions produce different communication patterns and should therefore be mapped to links with appropriate bandwidth and latency characteristics.

Intra-node interconnects: NVIDIA NVLink provides direct high-bandwidth GPU-to-GPU communication, while NVSwitch connects multiple GPUs through a switched fabric. For example, SXM-based A100 and H100 systems provide aggregate per-GPU NVLink bandwidths of approximately 600 GB/s and 900 GB/s, respectively. These interconnects are well suited to tensor parallelism because tensor-parallel collectives occur frequently within individual Transformer layers.

Inter-node network fabrics: InfiniBand and Remote Direct Memory Access over Converged Ethernet (RoCEv2) provide high-throughput communication between compute nodes. Depending on the adapter and network generation, individual links commonly operate between 100 Gbit/s and 400 Gbit/s. These fabrics support pipeline point-to-point transfers and collective operations used by DDP, ZeRO, and FSDP. Their performance also depends on switch topology, network contention, routing, and available bisection bandwidth.

PCI Express host interconnects: PCI Express connects GPUs to host processors and is frequently used for transfers between GPU and CPU memory. Approximate unidirectional bandwidth for a full $x16$ link is 16 GB/s for PCIe Gen3, 32 GB/s for PCIe Gen4, and 64 GB/s for PCIe Gen5. These rates are substantially lower than modern GPU memory bandwidth and high-bandwidth intra-node GPU interconnects. As a result, parameter or optimizer-state offloading can stall GPU execution unless transfers are prefetched and overlapped with independent computation.

Topology-aware placement typically maps communication-intensive tensor parallelism to the fastest intra-node links, pipeline stages to neighboring nodes with sufficient point-to-point bandwidth, and data-parallel replicas across the broader cluster. Failure to account for this hierarchy can place high-frequency collective operations on slower links and substantially reduce scaling efficiency.

Building upon these foundational parallelism and communication mechanisms, the following section examines recent systems that jointly optimize memory footprint, execution schedules, and cross-node communication.

IV. Research Advances & Synthesis

Emeerging research systems move beyond static, manually tuned parallelization topologies toward dynamic, overlap-aware, and hardware topology-optimized execution frameworks. These advanced systems automatically explore non-trivial execution schedules, hide memory movement latency, and compensate for dynamic network bottlenecks.

A. System Analysis 1: Mist

The Mist framework [1] introduces memory, overlap, and imbalance-aware automatic co-optimization for LLM distributed training. Prior auto-parallelism planners (e.g. Alpa, Aceso, AdaPipe) tuned parallelism grids while keeping memory configurations fixed, or limited their optimization to single techniques such as static activation recomputation. Consequently, baseline planners suffered from multile severe failure modes:

1) Alpa failed to find feasible solutions on memory-constrained L4 GPUs due to its memory-unaware intra-operator pass causing OOM errors.
2) Aceso underperformed Megatron-LM in 6 out of 10 evaluation cases because it lacked sharded data parallelism and communication-computation overlap awareness.

Mist introduces a fine-grained 4-stream overlap schedule teamplate that orchestrates CUDA execution streams to eliminate critical-path stalls:

1) Fine-Grained Four-Stream Overlap Scheduling

Mist introduces a fine-grained four-stream overlap schedule that coordinates computation, collective communication, and CPU–GPU data transfers. The objective is to hide communication and swapping latency behind independent GPU computation, thereby reducing stalls on the critical execution path. Table III summarizes the role of each CUDA stream during the forward and backward passes.

TABLE III

Mist four-stream overlap schedule

CUDA stream	**Forward-pass operation**	**Backward-pass operation**
Stream 1: Compute	Forward computation F_i	Backward computation B_i
Stream 2: NCCL	Parameter All-Gather for P_{i+1}	Gradient Reduce-Scatter for R_{i-1}
Stream 3: D2H	Activation offload, $A_i \rightarrow$ host memory	Gradient offload, $G_i \rightarrow$ host memory
Stream 4: H2D	Parameter swap-in, $P_{i+1} \leftarrow$ host memory	Activation swap-in, $A_i \leftarrow$ host memory

The compute stream executes the forward or backward kernels for the current layer or pipeline stage. Concurrently, the NCCL stream performs collective communication for neighboring operations. The device-to-host (D2H) stream

offloads tensors that are not immediately required, while the host-to-device (H2D) stream prefetches tensors before their next use. Effective overlap requires transfers to begin early enough to complete before the corresponding tensor becomes a dependency of the compute stream.

Mist also controls the fraction of each model-state category that is offloaded. Let

$$O_P,\ O_G,\ O_O,\ O_A \in [0, 1] \tag{19}$$

denote the offloading ratios for parameters, gradients, optimizer states, and activations, respectively. These controls can be represented as the vector

$$\mathbf{O} = \begin{bmatrix} O_P & O_G & O_O & O_A \end{bmatrix} \in [0, 1]^4. \tag{20}$$

For a state category X with total size M_X, an offloading ratio O_X corresponds to

$$M_X^{\text{host}} = O_X M_X, \qquad M_X^{\text{device}} = (1 - O_X) M_X. \tag{21}$$

Thus, $O_X = 0$ retains the entire state category in GPU memory, whereas $O_X = 1$ offloads the entire category to host memory when it is not required for computation. Intermediate ratios allow Mist to trade GPU-memory savings against PCIe transfer overhead and the available opportunities for overlapping those transfers with computation.

To evaluate millions of parallel combinations without executing concrete graph simulations, Mist employs a Symbolic-Based Efficient Performance Analysis System:

1) Symbolic Shape Execution & Tracing: Inputs, model dimensions, and parallel degrees are declared as abstract symbolic expressions (b, s, h, d, TP, PP, DP). Mist executes a single symbolic simulation pass using PyTorch fake tensors and meta-devices to build a symbolic computation graph.
2) Batched Value Substitutions: Analytical runtime and peak memory usage are derived as symbolic parametric equations. Candidate plans are evaluated via low-latency symbolic matrix substitutions rather than concrete operator executions, accelerating search space exploration by over $10^5\times$.
3) Data-Driven Multi-Op Interference Model: Accurately quantifies performance degradation when compute kernels, NCCL collectives, D2H transfers, and H2D transfers execute concurrently over shared buses.

Empirical telemetry verifies that Mist's symbolic performance analyzer achieves outstanding accuracy, recording an average runtime error ratio of 1.79% and an average memory footprint error ratio of 2.10%.

To solve execution imbalance across pipelien microbatches, Mist implements imbalance-Aware Hierarchical Tuning. This employs Intra-Stage Tuning and Inter-Stage Turning. Intra-Stage Tuning formulates a dual-objective optimization problem sampling a Pareto frontier across stable microbatch runtime (h_X) and execution time derivations (i_X) caused by offloading or sharding operations. Inter-Stage Tuning constructs a Mixed Integer Linear Programming (MILP) model solved via CBC, incorporating microbatch imbalance terms to find globally optimal pipeline partitions (W_X), recomputation layers (k_X), and rank asssignments:

$$\mathcal{L}_{\text{steady}} = (m-1) \max_{1 \le x \le p} h_x, \tag{22}$$

$$\mathcal{L}_{\text{fill/drain}} = \sum_{x=1}^{p} h_x, \tag{23}$$

$$\mathcal{L}_{\text{imbalance}} = \max_{1 \le x \le p} \left(i_x - \sum_{u=1}^{x-1} h_u \right), \tag{24}$$

$$\min \ \mathcal{L} = \mathcal{L}_{\text{steady}} + \mathcal{L}_{\text{fill/drain}} + \mathcal{L}_{\text{imbalance}}. \tag{25}$$

Here, h_x represents the execution latency of a stable microbatch at pipeline stage x, while i_x denotes the execution-time deviation of the first or last microbatch. This deviation accounts for operations such as parameter gathering, gradient reduction, tensor offloading, and tensor prefetching that are not present in steady-state microbatch execution. The variable m denotes the total number of microbatches processed during one training iteration.

2) Cumulative Performance Improvements

Mist's performance improvement results from progressively combining flexible activation checkpointing, fine-grained CPU offloading, and imbalance-aware pipeline scheduling. Table IV shows the cumulative speedup contributed by these optimizations relative to the Megatron-LM 3D-parallel baseline.

TABLE IV

Cumulative speedup breakdown for Mist

System configuration	**Cumulative speedup**	**Incremental increase**
Megatron-LM 3D-parallel baseline	1.00×	—
Baseline with flexible activation checkpointing	1.12×	+0.12×
Previous configuration with fine-grained CPU offloading	1.19×	+0.07×
Previous configuration with inter-microbatch imbalance-aware pipelining	1.28×	+0.09×

Across the evaluated GPU clusters, Mist achieves an average speedup of **1.28**×, with a maximum speedup of **1.73**×, relative to Megatron-LM. Compared with Aceso, Mist achieves an average speedup of **1.27**× and a maximum speedup of **2.04**× [1].

At a larger evaluated scale, consisting of an 80-layer GPT-3 model trained on 32 NVIDIA L4 GPUs, Mist achieves a speedup of up to **1.32**×. The relative benefit is greatest in memory-constrained and PCIe-constrained L4 environments, where fine-grained offloading and computation–communication overlap can hide part of the host–device transfer cost. In higher-bandwidth A100 environments connected through NVLink and InfiniBand,

the baseline already operates closer to the available hardware limits. Consequently, Mist has fewer remaining communication and data-transfer stalls to eliminate, producing smaller relative speedups.

B. System Analysis II: PGCS

Parallel Gradient Computation and Synchronization (PGCS) targets distributed training of multiple expert language models across geographically separated data centers. These expert models may later be combined through a mixture-of-experts architecture or model-merging technique. Compared with communication inside a data center, cross-data-center communication is constrained by lower bandwidth, longer propagation delays, and heterogeneous network connectivity. Consequently, global gradient synchronization can become a dominant component of the total training time [2].

PGCS addresses this problem through three principal mechanisms.

1) **Worker-group and parameter-server architecture.**

   Each data center contains worker groups and parameter servers with separate responsibilities. Worker groups compute gradients on distinct partitions of the local training data. Within each worker group, PGCS can combine pipeline parallelism between hosts and tensor parallelism within a host. Parameter servers aggregate gradients, perform parameter updates, and distribute updated parameters to the worker groups.

   Gradient synchronization occurs at two levels. First, parameter servers aggregate gradients from worker groups within the local data center. Second, parameter servers in different data centers perform global synchronization. After the global gradients have been obtained, each parameter server updates the model and distributes consistent parameters to its local workers.

2) **Parallel gradient computation and synchronization.**

   PGCS trains expert models according to a predefined alternating order. When one expert model completes gradient computation, its gradients enter the synchronization process. During that synchronization, the worker groups begin computing gradients for another expert model. Conceptually, the schedule overlaps

$$\text{Sync}(E_j) \parallel \text{Compute}(E_{j+1}), \tag{26}$$

   where E_j and E_{j+1} denote consecutively scheduled expert models. The expert models share the same architecture, allowing workers to switch between experts by changing parameter values rather than loading a different model structure. PGCS also adjusts local batch sizes according to the computing and communication capabilities of each data center.

3) **Bisection and linear-programming algorithm.**

   PGCS introduces BLP, an optimization algorithm that combines bisection search with linear programming. BLP selects a gradient-synchronization strategy under arbitrary data-center connectivity, per-link bandwidth limits, and link propagation delays.

Let T_g denote the global gradient-synchronization latency. BLP applies bisection search to progressively narrow the feasible interval

$$T_g \in \left[T_{\text{lower}}, T_{\text{upper}}\right]. \tag{27}$$

At each candidate value of T_g, a linear program determines whether gradient partitions, spanning-tree paths, and link-bandwidth allocations exist that satisfy all connectivity and capacity constraints. A feasible solution lowers the upper bound, whereas an infeasible solution raises the lower bound. This process continues until the interval is smaller than a specified tolerance.

Together, these mechanisms reduce accelerator idle time by overlapping the synchronization of one expert model with the gradient computation of another. However, PGCS assumes a predefined expert-training order and structurally uniform expert models. Its evaluation also treats network resources and topologies as known inputs, leaving dynamic WAN congestion, link failures, and rapid bandwidth variation as open challenges.

C. System Analysis III: Production-Scale Systems and Survey Synthesis

Recent surveys organize the requirements of distributed LLM training around three system properties: scalability, efficiency, and reliability (SER) [3]. These properties are interdependent. Increasing cluster size can expose communication bottlenecks and raise the probability of component failures, while techniques that improve memory efficiency can introduce additional communication or recomputation.

TABLE V

SCALABILITY, EFFICIENCY, AND RELIABILITY FRAMEWORK

Property	**System objective**	**Representative techniques**
Scalability	Expand training to large accelerator clusters while preserving correctness and useful scaling efficiency.	Hybrid parallelism, ZeRO/FSDP sharding, expert parallelism, and topology-aware placement.
Efficiency	Maximize accelerator utilization while controlling computation, communication, and memory overhead.	Communication–computation overlap, kernel fusion, optimized attention, memory offloading, and collective scheduling.
Reliability	Detect, isolate, and recover from hardware, network, and software failures without substantial loss of training progress.	Asynchronous checkpointing, anomaly detection, fault isolation, in-memory recovery, and checkpoint-free recovery.

1) Representative Efficiency Techniques

The survey literature identifies several systems that address different components of the SER framework [4].

CO2: CO2 seeks to overlap the forward pass, backward pass, gradient All-Reduce, and optimizer update. It uses a dual-buffer pipeline to execute the computation of one microbatch concurrently with communication associated with another microbatch. Separate CUDA and NCCL streams provide nonblocking execution. Although this design can reduce exposed communication latency, highly asynchronous execution can introduce race conditions if tensor dependencies are not enforced correctly.

EDiT: EDiT is based on local stochastic gradient descent. It performs local updates within workers and uses hierarchical gradient aggregation for synchronization across workers. Its adaptive policy changes the synchronization interval according to observed gradient variance. Longer local-update intervals reduce communication frequency but may increase divergence between local models and adversely affect convergence.

EST: EST uses a dynamic subnet-evolution process. During an exploration phase, it activates different parameter subsets and estimates their importance from gradient information. During the utilization phase, important parameters are retained while other active subnet parameters are rotated. By maintaining optimizer state only for active parameters and communicating gradients only for the active subnet, EST reports a 70% reduction in memory consumption and a 10× reduction in communication volume under its evaluated configurations.

Because these results are summarized through a survey, the original CO2, EDiT, and EST papers should be consulted before making detailed claims about their algorithms or experimental results.

2) Reliability at Production Scale

Production-scale training jobs may occupy thousands of accelerators for weeks or months. At this scale, failures are expected during a training run rather than treated as exceptional events. Reported failure sources include GPU errors, ECC errors, NVLink failures, NCCL timeouts, network connection errors, software failures, power interruptions, and cooling problems [3].

The infrastructure survey reports that hardware problems accounted for 78% of the failures observed during LLaMA 3 pretraining. It also reports that Alibaba's C4 analysis found approximately 82.5% of errors to be confined to a specific node or device. Even when a fault does not terminate the training job, link degradation and computational stragglers can lower Model FLOPs Utilization (MFU) because synchronous workers must wait for the slowest participant.

Traditional synchronous checkpointing pauses training while model state is copied and persisted to storage. As model size and accelerator count increase, this pause can become a substantial source of lost training time. Asynchronous checkpointing instead overlaps snapshot and persistence operations with subsequent training computations. A conceptual asynchronous checkpoint pipeline can be written as

$$\text{Train}(t+1) \parallel \text{Snapshot}(t) \parallel \text{Persist}(t-1), \tag{28}$$

where the current training iteration proceeds while recent state is copied to host memory and an earlier snapshot is written to persistent storage. This approach reduces checkpoint stalls but requires careful consistency control to ensure that a recoverable snapshot corresponds to a valid training state.

The SER framework therefore emphasizes that scalability cannot be evaluated only through peak throughput. A production training system must also maintain high utilization during failures, recover without excessive rollback, and prevent communication or storage operations from repeatedly blocking the training critical path.

V. Scope, Critique & Challenges

No single distributed training framework eliminates all computational, memory, and network bottlenecks simultaneously. High-performance software design rely on key structural assumptions that introduce distinct trade-offs under real-world production conditions.

A. Rigorous System Critique and Architectural Limitations

1) Critique of Mist (EuroSys '25)

Mist's intra-stage perofmrnace model assumes identical Transformer layer execution blocks within a pipeline stage. While valid for standardized decoder architectures (e.g. GPT-3, LLaMa), this assumptions breaks down on non-uniform models, such as Mixture-of-Experts (MoE) architectures with sparse expert layer placements, or hybrid models combining state-space matrix (e.g. Mamba) with attention blocks. Furthermore, when aggressive host memory offloading ratios (O_P, O_G, O_O, O_A) are selected on PCIe-based clusters (e.g., NVIDIA L4 nodes), the concurrent execution of Stream 3 (D2H) and Stream 4 (H2D) rapidly saturates bidirectional PCIe bus bandwidth. PCIe Interconnect & Host Bus Contention Mode Concurrent activation swap-out (Stream 3 D2H) and parameter swap-in (Stream 4 H2D) create heavy memory traffic across host CPU DRAM channels. This contention induces NUMA node memory thrashing and CPU cache line invalidation, causing execution latency spikes that invalidate Mist's analytical interference model predictions.

2) Critique of PGCS (IEEE TNSE)

PGCS relies on a static alternating expert computation schedule optimized using Bisection Linear Programming (BLP) network flow solutions. On production cross-site WAN links, dynamic IP routing changes, transient packet loss, and bursty background traffic introduce unexpected network jitter. Because PGCS lacks runtime dynamic re-pipelining, a single transient bandwidth drop on one WAN link can stall all parameter servers, destabilizing synchronous gradient updates across nodes.

B. System Scalability and Interconnect Saturation Barriers

As distributed training expands to thousands of accelerators across many physical nodes, scaling efficiency is increasingly constrained by the underlying communication infrastructure. Collective operations such as the parameter All-Gather operations used by ZeRO-3 and FSDP, as well as the All-to-All token exchanges used

by mixture-of-experts models, can generate substantial cross-rack traffic. When the network topology provides insufficient bisection bandwidth, simultaneous collective flows compete for the same uplinks and switch resources. This contention increases queueing delay and may produce packet loss, retransmissions, or collective-operation timeouts. Fat-tree and rail-optimized network topologies can provide greater aggregate bandwidth, but they require additional switches, optical transceivers, cabling, power, and careful topology-aware placement.

Synchronous training also amplifies the effect of stragglers because every rank must reach a synchronization point before the next training step can proceed. If t_i denotes the local execution time of rank i across N participating ranks, the effective step time is bounded by the slowest participant:

$$t_{\text{step}} = \max_{1 \le i \le N} t_i. \tag{29}$$

Consequently, thermal throttling, network congestion, transient PCIe retransmissions, operating-system interference, or gradual hardware degradation on one accelerator can delay the entire training group. As the number of ranks increases, the probability that at least one rank experiences a transient slowdown also increases, making straggler detection and mitigation essential for maintaining cluster-wide utilization.

Memory optimization introduces an additional trade-off between model capacity and communication cost. Moving from replicated or partially sharded training to full parameter sharding, as in ZeRO-3 or FSDP, reduces the static model state stored by each device. However, fully sharded execution must reconstruct parameters through All-Gather operations during the forward and backward passes. Under the simplifying assumption that parameter and gradient tensors have equal sizes, ZeRO-3 communicates approximately 1.5× the per-device byte volume of DDP or ZeRO-2, corresponding to a 50% increase in algorithmic communication volume. On bandwidth-constrained networks, the additional tensor transfers may outweigh the throughput gained by fitting a larger model or larger microbatches into GPU high-bandwidth memory. Effective scaling therefore requires jointly selecting the sharding strategy, microbatch size, parallelism dimensions, and placement topology rather than optimizing memory consumption independently of communication.

C. Open Research Directions

The preceding analysis identifies three open problems in distributed LLM training:

1) **Network-aware MoE routing.** Mixture-of-experts systems operating across heterogeneous networks require routing policies that adapt token placement to changing bandwidth, congestion, and link availability without compromising convergence or expert-load balance.
2) **Verification of multi-stream execution.** Fine-grained schedules such as Mist's four-stream design rely on carefully ordered CUDA events. Compiler-assisted race detection and dependency verification could identify missing barriers and unsafe memory accesses before they cause silent corruption.

3) **Low-overhead elastic recovery.** Future systems should recover from node failures without rolling back the entire training cluster. Promising techniques include asynchronous differential checkpointing, rapid fault isolation, and dynamic re-sharding of lost model state across surviving ranks.

These challenges motivate topology-aware training systems that adapt communication, scheduling, and recovery policies while preserving throughput and correctness at scale.

VI. Conclusion

Co-optimizing memory footprints, communication latencies, and system reliability is essential for scaling Large Language Models past single-accelerator capacity bounds. Modern dynamic execution engines demonstrate that co-designing software scheduling with physical interconnect topologies unlocks significant performance gains over static baseline frameworks. By orchestrating fine-grained multi-stream overlap, state sharding, and symbolic performance modeling, architectures such as Mist achieve substantial throughput speedups while preserving strict computational fidelity. However, physical bandwidth ceilings, network jitter, and straggler amplification at extreme cluster scales present persistent infrastructure bottlenecks. Resolving these challenges will require next-generation distributed systems to integrate elastic failure recovery, adaptive cross-site routing, and formally verified concurrent execution streams.

References

[1] Z. Zhu, C. Giannoula, M. Andoorveedu, Q. Su, K. Mangalam, B. Zheng, and G. Pekhimenko, "Mist: Efficient distributed training of large language models via memory-parallelism co-optimization," in *Proceedings of the Twentieth European Conference on Computer Systems*, ser. EuroSys '25. ACM, Mar. 2025, pp. 1298–1316. [Online]. Available: http://dx.doi.org/10.1145/3689031.3717461

[2] H. Li, H. Jiang, J. Wu, G. Yang, and J. Zhang, "Parallel gradient computation and synchronization: Enhancing the efficiency of distributed training for llms," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 1962–1976, 2026.

[3] J. Duan, S. Zhang, Z. Wang, L. Jiang, W. Qu, Q. Hu, G. Wang, Q. Weng, H. Yan, X. Zhang, X. Qiu, D. Lin, Y. Wen, X. Jin, T. Zhang, and P. Sun, "Efficient training of large language models on distributed infrastructures: A survey," 2024. [Online]. Available: https://arxiv.org/abs/2407.20018

[4] F. Zeng, W. Gan, Y. Wang, and P. S. Yu, "Distributed training of large language models: A survey," *Natural Language Processing Journal*, vol. 12, p. 100174, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2949719125000500